Distributed Heat Diffusion Simulation Using MPI

COURSE	LANGUAGE	EXECUTION
Distributed Systems	**C++ + MPI**	**2+ physical laptops**

1 Project overview

Your task is to build a C++ application that simulates how temperature changes over a two-dimensional metal plate. First solve the problem on one laptop using a sequential program. Then develop an MPI version that divides the computation among multiple physical laptops communicating over a network.

> **Connection to Lab 4** This project directly extends Lab 4: A Parallel Data Structure. You will apply the same distributed 2D-grid and ghost-row (halo) exchange concepts to a real iterative numerical computation.

2 Mathematical problem and fixed input

Represent the metal plate as a grid of 2,000 rows × 2,000 columns (4,000,000 temperature values). Each value is stored as a double-precision floating-point number. All groups must use the same initial conditions and execute exactly 2,000 update iterations for the final benchmark.

Parameter	Required value / rule
Grid dimensions	N = 2,000 rows, M = 2,000 columns
Initial interior temperature	Different for every cell, 20.0 to 119.9 °C, from the formula below
Top boundary (row 0)	100.0 °C, including the two top corner cells
Left / right / bottom boundaries	20.0 °C; boundaries remain fixed
Number of iterations	2,000 synchronous iterations
Numerical type	double (64-bit floating point)

Interior cells ($1 \le i \le N-2$, $1 \le j \le M-2$) do not all start at the same value. Each one starts at its own temperature, computed from its global indices with 64-bit unsigned integers, so every rank can generate its own rows without any scatter or shared file:

```
h = ((uint64_t)i * 73856093ULL) ^ ((uint64_t)j * 19349663ULL)
T[i][j] = 20.0 + (h % 1000) / 10.0
```

The boundary rows and columns are not computed from this formula; they keep the fixed values in the table. Because every interior cell starts different, a missing or wrong ghost-row exchange changes the final grid. This is how the check in Section 6 detects exchange errors.

3 Temperature update rule

At every iteration, update each interior cell using the four neighboring values from the previous iteration:

```
new[i,j] = (old[i-1,j] + old[i+1,j] +
            old[i,j-1] + old[i,j+1]) / 4.0
```

Update interior cells only ($1 \le i < N-1$ and $1 \le j < M-1$). Keep all boundary values fixed. Calculate each new grid from an unchanged old grid; do not update the old grid in place during an iteration. After the iteration, swap the roles of the two grids.

4 Required implementation

A. **Sequential implementation.** Write a non-MPI program that generates the required grid, performs the 2,000 iterations, and records solver execution time.

B. **Distributed MPI implementation.** Partition the grid into contiguous blocks of rows. Assign each MPI rank responsibility for computing the interior cells in its assigned rows.

C. **Boundary exchange.** At each iteration, exchange the necessary edge rows with neighboring MPI ranks. Store received values in ghost rows, so calculations at partition boundaries use the correct values from the preceding iteration.

D. **Synchronous updates.** Ensure each rank completes the required neighbor-data exchange and updates its region consistently. Use appropriate point-to-point and/or nonblocking MPI operations; avoid communication deadlocks.

E. **Final result and validation.** Gather the final grid (or the necessary parts of it) and verify that it agrees with the sequential implementation. Report a numerical correctness measure.

5 How the grid is distributed

For example, with two ranks, Rank 0 owns the first 1,000 global rows and Rank 1 owns the last 1,000. Each rank stores its owned rows plus the ghost row(s) required for communication. The actual partitioning must also be correct for a different number of ranks.

MPI rank	Owned global rows	Communication requirement
Rank 0	0–999	Send last owned interior-adjacent row; receive neighboring row into lower ghost region.
Rank 1	1000–1999	Receive neighboring row into upper ghost region; send first owned row upward.

The edge between the partitions is not a physical boundary of the metal plate. It is an artificial boundary introduced by distributing the data. Adjacent ranks must exchange values so the numerical result remains correct.

6 Correctness requirements

Identical problem: Both versions must use the same grid dimensions, initial conditions, boundary rules, and iteration count.

Numerical agreement: Compute the maximum absolute difference between corresponding cells of the sequential and MPI outputs:

```
max_error = max |T_sequential[i,j] – T_MPI[i,j]|
```

Acceptance criterion: Require max_error = 0 for the final grid. Both programs must compute each cell as (up + down + left + right) / 4.0 in that order and must not use -ffast-math (GCC) or /fp:fast (MSVC). A nonzero value must be explained in the report and must not exceed 1e−12 °C. If results differ, investigate indexing, ghost-row exchange, boundary handling, and update order.

Proof of distribution: Print each rank's number and the hostname where it runs (e.g., using MPI_Get_processor_name) and include evidence of execution on separate physical laptops.

7 Minimum MPI concepts to demonstrate

Your code must demonstrate MPI initialization/finalization, rank and communicator size, point-to-point boundary exchange, synchronization where needed, and a method to combine or validate final results. You may use MPI_Scatterv / MPI_Gatherv or an equivalent correct decomposition. You are responsible for designing the implementation; no complete solution is provided.

8 Multi-laptop deployment

Use at least two ordinary Windows laptops on the same reachable network. A dedicated server is not required. Students have administrator access to their own laptops, but must configure MPI remote launching and network permissions before benchmarking.

Software: C++ compiler and the same Windows MPI toolchain used in class (MS-MPI runtime, development headers/libraries, and mpiexec).

Execution: Build a compatible executable and ensure it is available at the required path on every host. Use a consistent working directory and application arguments.

Connectivity: Confirm that hostnames/IP addresses are reachable and that the network permits host-to-host traffic; some Wi-Fi networks isolate connected clients.

MPI service: Ensure the MS-MPI Launch Service (or another supported MS-MPI launching method) is configured on participating laptops and that Windows Firewall allows the required traffic.

Illustrative MS-MPI commands (replace hostnames and file paths to match your computers):

```
Local:    mpiexec -n 2 heat_mpi.exe
```

```
2 hosts:  mpiexec -hosts 2 LAPTOP_A 1 LAPTOP_B 1 -wdir C:\MPI_Project
heat_mpi.exe
```

On each Windows host, an administrator can start the launch service with: sc start MSMPILaunchSvc. The launch service must be installed and remote authentication must work; the commands alone do not guarantee connectivity.

9 Required benchmark configurations

Case	Physical laptops	Execution	Purpose
A	1	Sequential program (1 process)	Baseline runtime (Tseq)
B	1	MPI program, 2 ranks on the same laptop	Local parallel execution
C	2	MPI program, 1 rank per laptop	Mandatory multi-machine execution
D (optional)	3	MPI program, 1 rank per laptop	Additional scaling experiment

10 Measurement protocol

1. **Same workload.** Use 2,000 × 2,000 and 2,000 iterations in every final benchmark. Use an optimized build (e.g., -O2) consistently.
2. **Time the computation.** Use a high-resolution timer for the sequential solver and MPI_Wtime for the distributed solver. Exclude program startup, grid allocation, result printing, file I/O, and final output gathering from solver time. Include per-iteration communication and waiting.
3. **MPI timing boundaries.** Synchronize the ranks immediately before starting the timed iteration loop. Report distributed runtime as the maximum elapsed solver time among participating ranks (MPI_Reduce with MPI_MAX), not a single rank's time or the average.
4. **Repeat runs.** Perform one untimed warm-up, then at least three timed runs per configuration; report the median and individual run times. Keep laptops plugged in and minimize unrelated background activity.

Further reading / setup reference

Microsoft MPI — Running MS-MPI applications across multiple nodes:
https://github.com/microsoft/Microsoft-MPI/blob/master/docs/RunningMSMPI.md